Computability, Logic, and Randomness

Lecture Notes

Draft from October 29, 2026.
Check the website for the latest version.
Comments welcome!

Levin Hornischer
Levin.Hornischer@lmu.de
https://levinhornischer.github.io/CoLoR/

Contents

Preface 1

1. Introduction 4
1.1. Exercises 9

I. Computability 10

2. Formalizing computation: Turing machines 11
2.1. Turing machines 11
2.2. Church–Turing thesis 15
2.3. Enumerations and universal Turing machines 18
2.4. Why partial functions? 20
2.5. Exercises 21

3. Incomputability 23
3.1. The Halting problem 23
3.2. The Entscheidungsproblem 23
3.3. Rice's theorem 23
3.4. Tiling problems 23
3.5. Exercises 23

4. Turing degrees: Ordering problems by difficulty 24
4.1. Computably enumerable sets 24
4.2. Reducibility 24
4.3. Turing degrees 24
4.4. Exercises 24

II. Logic 25

III. Randomness 26

A. Appendix: set theory and order theory 27

Bibliography 30

Index 31

Preface

This is the reader for the course "Computability, Logic, and Algorithmic Randomness" given during the winter semester 2026/7 at *LMU Munich* as part of the *Master in Logic and Philosophy of Science*. These lecture notes are updated as the course progresses. A website with all the course material is found at

`https://levinhornischer.github.io/CoLoR/`.

Comments I'm happy about any comments: spotting typos, finding mistakes, pointing out confusing parts, or simply questions triggered by the material. Just email me at `Levin.Hornischer@lmu.de`.

Course description and objectives This course is an introduction to the theory around three interconnected concepts: computability, logic, and randomness. We start with the first concept: Computability theory describes which problems can be solved by an algorithm. This is made precise via so-called Turing machines, the theoretical underpinning of modern computers. Moving to the second concept, we see that not only can computability theory be used to answer questions in logic (e.g., prove Gödel's incompleteness theorem), but logic can be also used to informatively describe computational problems. Finally, time permitting, we look at the third concept: namely, randomness and its interplay with computability. We ask whether access to randomness helps computing power, or if purely random information is not useful information.

Prerequisites A prerequisite is an introductory course in first-order logic (soundness and completeness theorems, basic model theory) and some familiarity with mathematics (basic probability theory and some order theory), including the ability of writing mathematical proofs.

Layout These notes are informal and partially still under construction. For example, there are margin notes to convey more casual comments that you'd rather find in a lecture but usually not in a book. Todo notes indicate, well, that something needs to be done. References are found at the end. Exercises are at the end of each chapter.

This is a margin note.

This is a todo note

Study material Computability theory is a classic topic, so there are many textbooks available. Here are some:

There also is a discussion on math stackexchange on textbooks for computability theory.

- H. Rogers Jr. (1967). *Theory of Recursive Functions and Effective Computability*. New York: McGraw–Hill Book Company
- N. Cutland (1980). *Computability: An Inroduction to recursive function theory*. Cambridge: Cambridge University Press
- P. Odifreddi (1990). *Classical Recursion Theory: The Theory of Functions and Setsof Natural Numbers*. Studies in Logic and the Foundations ofMathematics, vol. 125. Amsterdam: North-Holland Publishing Company
- S. B. Cooper (2004). *Computability Theory*. Boca Raton,FL: Chapman & Hall/CRC
- G. S. Boolos et al. (2007). *Computability and Logic*. 5th ed. Cambridge: Cambridge University Press
- R. I. Soare (2016). *Turing Computability: Theory and Applications*. Berlin, Heidelberg: Springer. DOI: https://doi.org/10.1007/978-3-642-31933-4
- B. Robič (2020). *The Foundations of Computability Theory*. Berlin Heidelberg: Springer. DOI: 10.1007/978-3-662-62421-0
- G. Tourlakis (2022). *Computability*. Cham: Springer. DOI: 10.1007/978-3-030-83202-5

They also stress the tight connection with logic. For more, you can also consult Peter Smith's helpful "Beginning Mathematical Logic: A Study Guide", section 6 on "Arithmetic, computability, and incompleteness". There also are many lecture notes; in particular, I recommend Marks (2024).

For a category-theoretic take on computability: Yanofsky (2022).

For algorithmic randomness, the classic reference is

- R. G. Downey and D. R. Hirschfeldt (2010). *Algorithmic Randomness and Complexity*. New York: Springer

and especially for the theme on interactions between randomness and computability, see

- A. Nies (2009). *Computability and Randomness*. Oxford: Oxford University Press

Notation Throughout, 'iff' abbreviates 'if and only if'. A review of basic set theory and order theory that we use is found in appendix A.

1. Introduction

In this short introductory chapter, we motivate and overview the topics that we will cover in this course.

Computability theory studies the question: Which problems can be solved by a computer? Synonyms for 'computer' here are 'algorithm' or 'effective procedure'. To give a rigorous answer, the vague terms first have to be made precise (i.e., be *explicated* or *rationally reconstructed*, as philosophers would say). This will be done as follows.

1. A *problem* will be formally analyzed as a partial function $f : \mathbb{N} \rightharpoonup \mathbb{N}$: an input n encodes an instance of the problem and $f(n)$ encodes its solution, if it exists.
2. A *computer* will be formally analyzed as a Turing machine.
3. A computer *solving* a problem will be formally analyzed as the Turing machine computing a function which is identical to the function describing the problem.

Why partial functions and not, as usual, total functions? Intuitively, not every problem instance might have a solution. But there also formal reasons, which we discuss in section 2.4.

Historically, what gave rise to these questions was Hilbert's program. This program was Hilbert's response to the foundational crisis that mathematics faced at the beginning of the 20th century. To put mathematics on solid foundations, the suggestion was to fix a set of axioms and inference rules such that all—and only all—theorems of mathematics are derived from these axioms with those rules, ideally in an automated way. In the above terminology, we first enumerate all the mathematical statements and then the problem is: Given the input n, output 0 or 1 depending on whether the n-th statement is a theorem.

This is a very rough sketch of Hilbert's program; for more accuracy and details, see Zach (2023).

You might reflect on similarities and dissimilarities to the turmoil in mathematics caused by AI, now about a century later.

Gödel's incompleteness theorem famously shattered Hilbert's program, and together with the work of Turing, Church, and others computability theory was started. We now sketch the main insights, before doing this formally and in detail in the next chapters.

Problems and solutions As mentioned, a problem is formalized as a partial function $f : \mathbb{N} \rightharpoonup \mathbb{N}$. This may seem quite restricted. Indeed, this does not include problems like 'What is the meaning of life?'. But

it includes much more problems than it might initially seem. It includes all 'discrete' problems, i.e., those for which each problem instance and its solution can be described symbolically—like in Hilbert's case above. This is because each symbolic problem instance and each possible solution can uniquely be encoded by a natural number, so the problem becomes a partial function $f : \mathbb{N} \rightharpoonup \mathbb{N}$ which, on input n, is defined iff n encodes a problem instance, in which case $f(n)$ encodes the solution. But what would it mean to solve such a problem with a computer?

Turing machines In a remarkable feat of mathematical philosophy, Alan Turing explicated in 1936 the intuitive notion of (symbolic) computation with an abstract machine, now known as the *Turing machine*. It processes symbols, that are written on a tape, according to its program, which is simple a table of instructions. At any given time step, the machine is in some internal state and positioned at some cell of the tape. Based on its internal state and the symbol written in that cell, the program says (a) in which internal state the machine should go into next, (b) which symbol the machine should write into the present cell instead, and (c) whether it should move to the left or the right cell next. The machine has one designated internal state known as the halting state: if it enters that, it stops its computation.

The Turing machine then computes the following function: It takes as input a string of symbols (consisting only of symbols that also the machine uses). They are written on the otherwise blank tape. Then the machine start processing according to its program. If it enters its halting state, then the string of symbols that is written on the tape at that moment is the output of the function. If the machine never enters a halting state and 'loops forever', then no output is ever given.

Church–Turing thesis Why should that be 'the correct' way of formalizing computation? Other models of computation have been developed: e.g., Church's lambda calculus or Gödel's recursive functions. Remarkably, though, they all turned out to be equivalent: a function is computable in one model if and only if it is computable in the other model. This has lead to the conviction that the formal concept of Turing-computability really coincides with the intuitive concept of (symbolic) computabilility. This is known as the *Church–Turing thesis*: that a function is Turing-computable if and only if it is computable in the intuitive sense. Note that this is a *philosophical* and not a mathematical claim. It is a claim about an informal

philosophical concept: namely, that of computability. It cannot be proven by formal mathematical means.

Methods to establish (non-) computability Having established a formalization of our initial question—namely, which problems are solvable by a computer?—, the next question is how we can establish whether or not a problem is computable.

The expected functions indeed turn out to be computable. For example, the addition function that maps a code for a pair of numbers (n, m) to the number $n + m$. Similarly for multiplication, exponentiation, ordering, etc. But Turing already was also asking: are there functions that are not computable?[1]

In other words, are there things that no computer can ever solve? Turing considered the so-called *halting problem*: Given a Turing machine e and an input n, decide if the Turing machine on this input halts or not. A Turing machine that solves this problem is a 'meta-program' that checks whether an inputted programs ever gets stuck in a loop. So it would be very desirable to have such a meta-program, but Turing showed that this cannot exist.

Theorem 1.1 (Turing)**.** *There is no Turing machine that solves the halting problem. In other words, the* h *function that maps a (code of a) Turing machine* e *and an input* n *to*

$$h(e, n) := \begin{cases} 1 & \textit{Turing machine } e \textit{ halts on input } n \\ 0 & \textit{otherwise} \end{cases}$$

is not computable.

Proof. Toward a contradiction, assume h is computable. Then we can define the Turing machine e that, on input n, computes $h(n, n)$ and if this is 0 it outputs 0 and otherwise loops forever. Now, we plug e into h. Since h always outputs 0 or 1, there are two cases:

- Case 1: $h(e, e) = 0$. Then, by definition, the Turing machine e with input e outputs 0, i.e., halts, so $h(e, e) = 1$.
- Case 2: $h(e, e) = 1$. Then, by definition, the Turing machine e with input e runs forever, i.e., does not halt, so $h(e, e) = 0$.

[1] Turing machines are finite objects, there are only countably many Turing machines, so there are also only countably many computable functions. On the other hand, there are uncountably many function $\mathbb{N} \to \mathbb{N}$ (by Cantor's diagonalization theorem). So there must be *some* uncomputable functions. But explicitly providing one is more difficult.

Thus, we indeed get a contradiction. □

Knowing the non-computability of the halting problem is a stepping stone for other non-computability results. A common method to prove a given problem to be non-computable is a *reduction argument*: assume there is a Turing machine solving the given problem and the use this to build a Turing machine that solves the halting problem. Since the latter is impossible, the former is, too.

This is how the historically important *Entscheidungsproblem* was answered in the negative: there is no algorithm to decide whether a given formula in first-order logic is logically valid (i.e., true in all interpretations). If there was such an algorithm, we could solve the halting problem: given Turing machine e and input n, there is a clever way to build a formula φ of first-order logic such that φ is valid iff e halts on input n, so if we can decide validity we can also decide halting.

In fact, also Gödel's first incompleteness theorem can be proven by a reduction argument. (Gödel's original proof was different: Turing machines did not exist yet at that time.)

The formulation that we state here uses the concept of a computable enumeration. A set of string is *computably enumerable* if there is a Turing machine that, as it keeps processing, writes on the tape strings separated by a blank cell such that all written strings are in the set and each string of the set eventually is written on the tape. This is weaker than the set of string being *decidable*: for that we require a Turing machine that take a string as input and outputs 'yes' or 'no' depending on whether or not the string is in the set. In the computably enumerable case, we only get confirmation if the string is in the set. If the string is not in the set, then, as we follow the enumeration, we never know if the string actually is in the set but has not yet appeared or actually is not in the set and because of that did not appear. On the other hand, in the decidable case, we also get confirmation if the string is not in the set.

This version is discussed, e.g., in this online lecture.

Theorem 1.2 (Gödel's first incompleteness theorem)**.** *There is no computably enumerable list of axioms that prove all and only the true statements of elementary mathematics (including, say, basic arithmetic, finite combinatorics, etc.).*

Just arithmetic is enough, because other 'elementary mathematics' can be encoded with natural numbers, so questions there reduce to questions about arithmetic.

Proof. For a reduction argument, assume there is such a list of axioms. By systematically applying all the inference rules, we can computably enumerate all and only the theorems of the theory (not just the axioms). But then we can solve the halting problem: Given a Turing machine e and an input n, consider the statement 'Turing machine e halts on input n'.

This is a statement of elementary mathematics. So either this statement or its negation shows up on the enumeration of all theorems (one of the two has to). So we start the enumeration and wait until one of the two shows up, and depending on which, we say the Turing machine halts or does not halt. This hence would be a computable procedure to solve the halting problem, which is impossible. □

This shattered Hilbert's dream of writing down, in a systematic—and hence computably enumerable—way all the axioms of mathematics, so that every mathematical truth can be immutably established by deriving it from the axioms.

Turing degrees By now, we have seen computable problems as well as non-computable problems, and the method of reducing one problem to another. This invites the general question: What structure does the collection of all problems have, ordered by reduction? This is the structure of the Turing degrees. It measure the 'degree of unsovability' of problems. The higher up a problem is in this order, the harder it is to solve. Hence much research went into understanding this order, with stunning results.

For example, every countable partial order embeds into the order of Turing degrees, so they are, in a sense, as complex as can be. Also, the first-order theory of the Turing degrees is very complicated: it is as hard as true second-order arithmetic. Finally, the answer to Post's problem shows that there are degrees strictly between the computable problems and the halting problem.

Randomness So far, we have seen interactions between computability and logic. But what about the third concept in the course's title? Roughly, randomness is the polar opposite of computability: Consider the outcome of an infinite sequence of coin tosses $f : \mathbb{N} \to \{0, 1\}$, i.e., $f(n) = 0$ (resp., $= 1$) means that the n-th toss was heads (resp., tails). This sequence is random, so we should not be able to predict with some algorithm what comes next, i.e., the function f should not be computable. The field of algorithmic randomness studies when a sequence is random. Thus, by understanding randomness, we also gain insights about its opposite, computability.

More specifically, we study the relationship between computability and randomness by asking whether randomness is a resource to computability. If the Turing machine may use the outcome of a sequence of coin tosses in its computation, can it, for some nonzero probability of coin tosses, correctly solve the halting problem? The answer is no, which is a theorem

by de Leeuw, Moore, Shannon, and Shapiro and, independently, Sacks. Intuitively, then, while a single random sequence contains a lot of information (since it is random, we cannot compress it into any sequence with fewer information), they do not, on average, contain any 'useful' information with which we can compute more than what we could compute with deterministic Turing machines.

This is for computability in general. Randomness might still help a computing agent with finite memory: see Icard (2019).

However, we may push further and ask if this not only holds 'on average' but also for individual sequences: Is it true that random sequences contain a lot of information but no useful information? Here the answer is no for arguably the most popular formalization of randomness known as Martin-Löf randomness. One particular sequence, known as Chaitin's Ω, is Martin-Löf random but it can compute the halting problem. (And the Kučera–Gács theorem show that in fact every sequence can be computed from some Martin-Löf random sequence.) But this ceases to be the case for the stronger notions of randomness (like difference randomness or weak 2-randomness). So the interplay between computability and randomness is rather delicate!

1.1. Exercises

Exercise 1.a. We said that computability theory consider 'discrete' problems $f : \mathbb{N} \rightharpoonup \mathbb{N}$ where problem instances and their solutions can be described symbolically. But what about computing with real numbers, i.e., computing functions $f : \mathbb{R} \rightharpoonup \mathbb{R}$? Can they be captured on the symbolic point of view (e.g., as floating point real numbers)? Or do we need a different theory?

For more on this, see, e.g., Pour-El and Richards 1989.

Exercise 1.b. Here we only considered abstract mathematical models of Turing machines. We do not consider what it means to implement or realize a Turing machine in a physical system, like my laptop. Think about what such a 'realization' relation between the theoretical model and a concrete system involves.

For more on this, see, e.g., Gandy (1980) and Sieg (2002).

Part I.

Computability

2. Formalizing computation: Turing machines

In this chapter, we introduce Turing machines to formalize the intuitive idea of computation.

to be updated

2.1. Turing machines

We now formalize the informal description of a Turing machine discussed in chapter 1. We fix the symbol $\textvisiblespace$ and call it the *blank symbol*.

Definition 2.1. A *Turing machine* M is a tuple $(S, Q, q_{\text{start}}, q_{\text{halt}}, \delta)$ where

- S is a finite set with $\textvisiblespace, 1 \in S$. We call S the *alphabet* of M, and we call the elements of S *symbols* of M. We call the elements of $S \setminus \{\textvisiblespace\}$ the *non-blank symbols* of M.
- Q is a finite set with $q_{\text{start}}, q_{\text{halt}} \in Q$. We call the elements of Q the (*internal*) *states* of M, we call q_{start} the *starting state*, and we call q_{halt} the *halting state*.
- $\delta : S \times Q \rightharpoonup S \times Q \times \{L, R\}$ is a partial function. We call δ the *transition map* or *program* of M.

A *configuration* (or *overall state*) of M is a how the machine can 'look like' at any given moment: precisely, it is a triple (σ, q, n) where

- $\sigma : \mathbb{Z} \to S$ is a function, with $\sigma(k) \in S$ being the symbol written at position $k \in \mathbb{Z}$ of the tape,
- $q \in Q$ is the internal state of the machine, and
- $n \in \mathbb{Z}$ is the position on the tape where the machine is located.

A *run* of M is a finite or infinite sequence

$$(\sigma_0, q_0, n_0), (\sigma_1, q_1, n_1), (\sigma_2, q_2, n_2), \ldots$$

of configurations of M such that

- The machine starts in the starting state at position 0, i.e., $q_0 = q_{\text{start}}$ and $n_0 = 0$, while σ_0 describes what is written on the tape at the start.

- For all i, if q_i is the halting state q_{halt}, then (σ_i, q_i, n_i) is the last entry of the sequence, and the runs ends in the halting state. If q_i is not the halting state, there are two cases. If the program $\delta(\sigma_i(n_i), q_i))$ is not defined for the currently read symbol and state, then the machine stalls but does not halt, and the run ends with (σ_i, q_i, n_i) but not in a halting state. If the program $\delta(\sigma_i(n_i), q_i)) = (s, q, X)$ is defined, then the next configuration $(\sigma_{i+1}, q_{i+1}, n_{i+1})$ must be such that $q_{i+1} = q$ and

$$n_{i+1} = \begin{cases} n_i + 1 & \text{if } X = R \\ n_i - 1 & \text{if } X = L \end{cases} \qquad \sigma_{i+1}(k) = \begin{cases} s & \text{if } k = n_i \\ \sigma_i(k) & \text{otherwise.} \end{cases}$$

Finally, the *partial function* f_M *computed by* M is defined as follows: Given a string x of non-blank symbols (i.e., x is a finite sequence of elements from $S \setminus \{␣\}$), start the machine with the configuration $(\sigma_0, q_{\text{start}}, 0)$, where σ_0 is blank everywhere except having written the string x starting at position 0; if the machine's run ends with a halting state, $f_M(x)$ is defined and the output is the string y that consists of the symbols to the left of the current position until the first blank symbol—otherwise, if the run does not end in a halting state, $f_M(x)$ is undefined.

There are many variants of this definition. For example, we could refrain from requiring the symbol 1, we could require also a symbol 0, we could have one fixed starting state and initial state (like the blank symbol, rather than making it part of the signature of a Turing machine), we could even allow several tapes, or instead of L and R we could allow also N for staying put on the present position of the tape. However, this would not change what the Turing machines are able to do: because one variant can simulate the other variant. For example, staying put (N) amounts to the same as first moving right (R) and then left (L). One might need a few more steps, but ultimately can achieve the same. Formally, they compute the same function, in the following sense.

See exercise 2.a.

Definition 2.2. A function $f : \mathbb{N}^k \to \mathbb{N}$ is (*Turing-*) *computable* if there is a Turing machine over symbols $S = \{␣, 1\}$ such that, for all $(x_1, \ldots, x_k) \in \mathbb{N}^k$, we have the following equivalence:

$f(x_1, \ldots, x_n)$ is defined and with value y

$\Leftrightarrow$ the function $f_M(\overline{x})$ computed by M is defined with value $\overline{y}$, where $\overline{x}$

is the string

$$\underbrace{1\ldots1}_{x_1 \text{ many}} \sqcup \underbrace{1\ldots1}_{x_2 \text{ many}} \sqcup \cdots \sqcup \underbrace{1\ldots1}_{x_k \text{ many}}$$

and $\overline{y}$ is the string

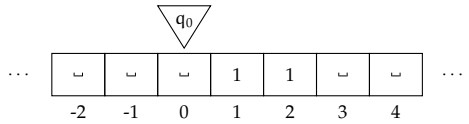
$$\underbrace{1\ldots1}_{y \text{ many}}.$$

Example 2.3. Consider the following Turing machine that computes the function $f : \mathbb{N} \to \mathbb{N}$ given by $f(x) = x + 1$, i.e., the successor function.

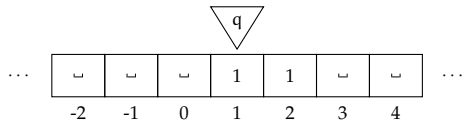
Let $S = \{\sqcup, 1\}$, let $Q = \{q_0, q_1, q\}$ with $q_{\text{start}} = q_0$ and $q_{\text{halt}} = q_1$, and let δ be given as follows:

S	Q	S	Q	{L, R}
$\sqcup$	q_0	$\sqcup$	q	R
1	q	1	q	R
$\sqcup$	q	1	q_1	R

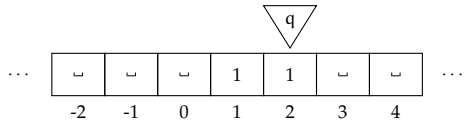
Let's see how the machine processes the input $x = 2$, for which it should produce the output $f(x) = 3$. The starting configuration looks as follows:



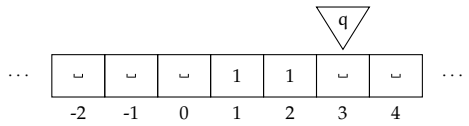
Hence the first row of the instruction table applies, so we write a blank, go into state q and move the head right. So the next configuration is



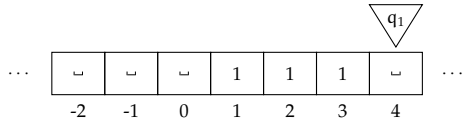
Now the second row of the instruction table applies, so we write a 1, go into (or rather stay in) state q, and move the head right. So the next configuration is



Now still the second row of the instruction table applies, so we write a 1, go into q, and move the head right. So the next configuration is

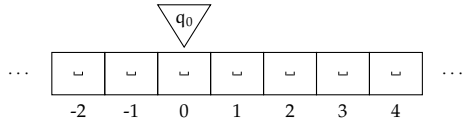


Now the third row of the instruction table is applicable, so we write a 1, go into state q_1, and move the head right. So the next configuration is

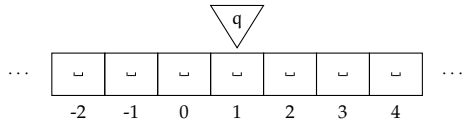


Now we have reached the halting state, so the run stops here in the halting state. We can read off the output: it is the string to the left of the current position until the first blank, i.e., 111. As needed, this represents the value $f(x) = 3$.

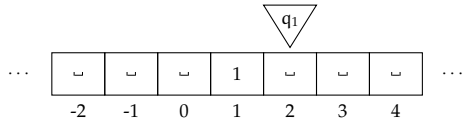
Other inputs work similarly. However, it is good to consider the 'edge case' $x = 0$. There, the starting configuration looks as follows:



Hence again the first row of the instruction table applies, so we write a blank, go into state q and move the head right. So the next configuration is



Now the third row of the instruction table is applicable, so we write a 1, go into state q_1, and move the head right. So the next configuration is



Now we have reached the halting state and the run stops. We read off the output, namely the string 1, which again represent the value $f(x) = 1$, as needed. ⌟

Exercise 2.b asks you to do more of such examples.

A final comment to avoid confusion: The definition of being computable means that *there exists* a Turing machine that computes the function. As

elsewhere in mathematics, this is a classical existence claim and not a constructive one. That is, we do not necessarily need to explicitly construct the Turing machine; it is enough to prove the such a machine must exist, even if we do not know exactly how it looks like.

To illustrate this point, consider your favorite currently unsolved mathematical conjecture; say, the Goldbach conjecture (every even integer greater than 2 is the sum of two primes). Consider the function $f : \mathbb{N} \to \mathbb{N}$ defined by

$$f(x) := \begin{cases} 1 & \text{Goldbach's conjecture is true} \\ 0 & \text{otherwise.} \end{cases}$$

Then there are two cases: either Goldbach's conjecture is true and f is the constant 1 function, or the conjecture is false and f is the constant 0 function. In either case, there is a Turing machine computing f. We just currently do not know which Turing machine computes f (otherwise we had solved the Goldbach conjecture); we merely know that there exists a Turing machine which computes f.

2.2. Church–Turing thesis

Now does the formalization given by Turing machines really capture our intuitive idea of computation? The claim that it does is now known as the Church–Turing thesis.

Thesis 2.4 (Church–Turing thesis)**.** *A function $f : \mathbb{N}^k \to \mathbb{N}$ is computable in the intuitive sense if and only if it is Turing-computable.*

When you see this for the first time, this might feel a bit odd. In math textbooks, one finds 'definition' or 'theorem', etc., but not 'thesis'. In particular, formal definitions in mathematics are not accompanied with theses that they capture the corresponding intuitive notion. As Marks (2024, p. 18) puts it: "there isn't a "Cauchy's thesis" asserting the ϵ-δ-definition of continuity is the correct definition of the intuitive idea of "continuous"." In particular, we cannot give a *formal* proof of the thesis, because this would require a formal definition of 'computable in the intuitive sense', which is precisely what we were looking for.

There are two main uses of the Church–Turing thesis:

1. It is useful for questions that ask whether a problem can be solved by providing an effective procedure—like Hilbert's Entscheidungsproblem, for example. Giving a positive answer amounts to providing

such a procedure for which everyone agrees that it is indeed effective. However, giving a negative answer is much harder. It requires to show that no possible procedure solves the problem. To show this, we need a formalization of 'effective procedure' to then be able to show that none of these now formalized mathematical objects has the property of solving the problem. This is the bottleneck to solving the Entscheidungsproblem in the negative (which we will see in section 3.2 below) that Turing overcame. By showing that there is no Turing machine that solves it, we can use the Church–Turing thesis to deduce that no effective procedure exists, and thus solving the original problem.

2. We can use the usual mathematical natural language to describe an effective procedure and then appeal to the Church–Turing thesis that there must also be a formal Turing machine which implements this procedure. This line of argument is very common in the field of computability theory, and we will already use this in the proof of theorem 2.8 below.

Note that both uses appeal to the left-to-right direction. The other direction is quite obvious: if a function is computable by a Turing machine, it is intuitively computable, because the Turing machine offers a clear procedure to follow.

Here we will, as is common in the field, assume that the thesis is correct. The most important 'evidence' for the thesis is its robustness: Many other formalizations of the intuitive notion of computation have been suggested and they are all extensionally equivalent, i.e., they count the very same functions as computable. Such other formalizations include: The lambda calculus, register machines, or recursive functions.

For more on the Church–Turing thesis, see Copeland (2026). For the idea and prospect of a 'squeezing argument', see Smith (2011).

In the remainder of this section, we consider recursive functions. We will not need them again in the course. But it still is helpful to see one very different approach to describing computable functions. This helps to really appreciate the robustness of the Church–Turing thesis, since the extensional equivalence is not obvious.

The idea of the recursive function approach to characterizing the intuitively computable functions is "axiomatic". One declares some basic functions to be recursive that are obviously computable, and then one allows some constructions that build new functions from old ones and that should also be considered computable. The formal definition is as follows.

Definition 2.5. The class of *partial recursive functions* is the smallest subset $\mathcal{C}$ of $\bigcup_{n\in\mathbb{N}}\{f : \mathbb{N}^n \rightharpoonup \mathbb{N}\}$ such that

1. The *successor function* $f(x) = x + 1$ is in $\mathcal{C}$. So f is a total function of one variable.
2. The *constant functions* $f(x_1, \ldots, x_n) = m$ are in $\mathcal{C}$, for $m, n \geqslant 0$. If $n = 0$, then f takes no argument and constantly outputs the value m.
3. The *projection functions* $f(x_1, \ldots, x_n) = x_i$ are in $\mathcal{C}$ for $1 \leqslant i \leqslant n$.
4. Closure under *composition*: If $h \in \mathcal{C}$ is a function of m variables and $g_1, \ldots, g_m \in \mathcal{C}$ are all functions of n variables, then the function $f(x_1, \ldots, x_n)$ is in $\mathcal{C}$ defined by (writing $\overline{x} = (x_1, \ldots, x_n)$):

$$f(\overline{x}) = h\big(g_1(\overline{x}), \ldots, g_m(\overline{x})\big).$$

5. Closure under *primitive recursion*: If $g \in \mathcal{C}$ is a function of $n \geqslant 0$ variables $\overline{y} = (y_1, \ldots, y_n)$ and $h \in \mathcal{C}$ is a function of $n + 2$ variables, then the function $f(x, \overline{y})$ of $n + 1$ variables is in $\mathcal{C}$ defined recursively by

$$f(x, \overline{y}) = \begin{cases} g(\overline{y}) & \text{if } y = 0 \\ h\big(x - 1, f(x - 1, \overline{y}), \overline{y}\big) & \text{if } y > 0 \end{cases}$$

6. Closure under μ-*recursion*: If $g(\overline{x}, y) \in \mathcal{C}$ is a function the $n + 1$ variables $\overline{x} = (x_1, \ldots, x_n)$ and y, then $f(\overline{x}) \in \mathcal{C}$, where
   - $f(\overline{x})$ is the smallest y such that both (a) $g(\overline{x}, y)$ is defined and equals 1 and (b) for all $y' < y$ also $g(\overline{x}, y)$ is defined but does not equal 1
   - $f(\overline{x})$ is undefined if no such y exists.

The smallest class of functions satisfying just 1–5 is called the class of *primitive recursive functions*.

There is a classic argument that there must be computable functions that are not primitive recursive: see exercise 2.f.

Now, one can show the equivalence to Turing-computability (e.g. Odifreddi 1990, thm. I.7.12):

> A function is partial recursive iff it is Turing-computable.

The left-to-right direction is fairly straightforward: One needs to show that the class of Turing-computable functions satisfies the axioms for partial recursive functions. This is exercise 2.d. The other direction is more

tedious, using the idea of 'arithmetization': we will not cover this here, but see, e.g., Odifreddi (1990, ch.I.7).

Gödel introduced the what we now call primitive recursive function in the proof of his 1931 incompleteness theorem. Also with Kleene's addition of clause 6 or equivalent characterizations, Gödel was not convinced that the notion of a recursive function captures the intuitive notion of computation. But he was convinced once he saw Turing's analysis: In 1946, writing for the Princeton bicentennial conference on problems in mathematics, Gödel says:

> ... with this concept [of Turing-computability] one has for the first time succeeded in giving an absolute definition of an interesting epistemological notion, i.e., one not depending on the formalism chosen. (Gödel 1946/1990, p. 150)

For a concise history of these ideas, see Soare (2016, ch. 17).

2.3. Enumerations and universal Turing machines

Because Turing machines are finite objects, there are only countably many of them and we can list them. This is a very important observation for computability theory, so let us state it more precisely.

A Turing machine is a tuple $(S, Q, q_{\text{start}}, q_{\text{halt}}, \delta)$. Since we are interested in those Turing machines that compute partial functions over $\mathbb{N}$, we restrict attention to those Turing machines with alphabet $S = \{␣, 1\}$. Without loss of generality, we can assume that S and Q are sets of the form $\{0, \dots, n\}$, otherwise just rename the symbols and states. In particular, we identify $␣$ with 0 and set qstart $= 0$ and $q_{\text{halt}} = 1$. So the data $(S, Q, q_{\text{start}}, q_{\text{halt}})$ are completely given by $S = \{0, 1\}$ and the number of states used. Regarding δ, one row of the table of instructions for a given S and Q is a tuple (s, q, s', q', x) with $x = 0$ standing for L and $x = 1$ standing for R. So we can order these possible rows lexicographically. So one δ is a list of rows in ascending order. Hence we can also list all possible δ's for given S and Q lexicographically. In sum, we can list all possible Turing machines $(S, Q, q_{\text{start}}, q_{\text{halt}}, \delta)$ lexicographically as $M_0, M_1, M_2, \dots$.

If we have symbols $a < b < c$, *the lexicographic order of tuples of length 3 is* $aaa, aab, aac, aba, abb, \dots$.

The details of doing this listing are not important. It only matters that it is done computationally: given n, we can compute what the n-th Turing machine M_n (as the n-th entry in our above lexicographic list). Historically, one uses the letter e here instead of n.

Definition 2.6. Let $(M_e)_{e\in\mathbb{N}}$ be a computable enumeration of all Turing machines with the alphabet $S = \{␣, 1\}$. We fix this enumeration once and for all and call it the *standard* (or *canonical*) *numbering*. We say e is the *index* of the *Gödel number* of machine M_e.

Given a machine M_e and a number k of variables, M_e computes a partial function $\mathbb{N}^k \rightharpoonup \mathbb{N}$, as described in definition 2.2. We denote this function $\varphi_e^{(n)}$. We write φ_e for $\varphi_e^{(1)}$. We also omit the superscript if it is clear form the variables: e.g., we write $\varphi_e(x, y)$ for $\varphi_e^{(2)}(x, y)$.

From now on, we identify the Turing machine M_e and the partial function φ_e with the index e. (So we say, e.g., Turing machine e does this or that, or partial function e is defined on $x = 3$, etc.)

Two comments: First, fixing another computable enumeration, we would get a different numbering from the one we declared to be the standard one. As mentioned, this does not really matter. The following theorem (which we will not prove) makes this precise:

> *The Acceptable Numbering Theorem*: Let $(\psi_e)_{e\in\mathbb{N}}$ be another numbering of the Turing-computable functions in one variable which is *acceptable*, i.e., there are computable functions $f, g : \mathbb{N} \to \mathbb{N}$ such that $\varphi_{f(n)} = \psi_n$ and $\psi_{g(n)} = \varphi_n$. Then there is a computable bijection $h : \mathbb{N} \to \mathbb{N}$ such that $\varphi_e = \psi_{h(e)}$ for all e.

Second, given a computable function, there are infinitely many Turing machines that compute it. This is known as the:

Lemma 2.7 (Padding lemma)**.** *Each computable function has infinitely many indices.*

Proof sketch. If M is a Turing machine that computes a given computable function, add unnecessary loops to M. Exercise 2.g asks you to make this proof sketch precise. □

The first and very important consequence of being able to enumerate Turing machines is the existence of universal Turing machines.

Theorem 2.8 (Universal Turing Machine)**.** *There is a Turing machine u such that, for all e and x, we have $\varphi_u(e, x) = \varphi_e(x)$. We call such u a universal Turing machine.*

Note that, by the padding lemma, there are finitely many universal Turing machines. We now use, for the first time, the Church–Turing thesis in a proof.

Proof. We argue in usual mathematical language that the function $(e, x) \mapsto \varphi_e(x)$ is computable. Then, by the Church–Turing thesis, there is a Turing machine u which computes that function, hence $\varphi_u(e, x) = \varphi_e(x)$, as needed.

To compute the function $(e, x) \mapsto \varphi_e(x)$ proceed in two steps: Given (e, x), first convert the number e into the corresponding Turing machine M_e (which can be done effectively since the numbering is computable). Second, simulate the machine M_e on the input x. Should the simulation halts, then stop and output the result of the simulation, and otherwise keep running the simulation. □

The conceptual importance of the existence of universal Turing machines is that we do not need one machine per problem, but there is a single machine that can solve all computable problems, because it can simulate the machines that solve that problem. Ultimately, this means that we can have one single laptop on which we can run all kinds of programs.

2.4. Why partial functions?

In mathematics, functions are total. So when we ask which functions are computable, we would like to characterize the functions $\mathbb{N}^k \to \mathbb{N}$ that are computable in the intuitive sense. In other words, we would like to define a list $(f_n)_{n\in\mathbb{N}}$ of functions $\mathbb{N}^k \to \mathbb{N}$ such that:

See Soare (2016, sec. 1.1.2).

1. The list $(f_n)_{n\in\mathbb{N}}$ contains all and only the functions that are intuitively computable.
2. There is a uniformly effective listing of them, i.e., an intuitively computable function g such that, for all n and x, we have $g(n, x) = f_n(x)$.
3. Every f_n is a total function.

However, we have now developed such a characterization for the more general *partial* functions. Conceptually this makes sense: the function that an algorithm computes can be partial if we allow algorithms that are not guaranteed to halt—which is very plausible. This way, we get the partial computable functions (φ_n), i.e., items 1 and 2 but not 3.

This still leaves the question: Could we restrict our attention to just the total functions that are computable by Turing machines, i.e., get all items 1–3? The answer is *no*, the three items are jointly inconsistent.

Also cf. the primitive recursive functions and exercises 2.e and 2.f.

The argument is a diagonal argument, which we will still see several times. We can define the diagonal function $h(x) := g(x, x) + 1$, which is intuitively computable because g is and $+$ is, but h cannot be on the list, otherwise, there is n such that $f_n = h$, but then

$$f_n(n) = h(n) = g(n, n) + 1 = f_n(n) + 1.$$

Since items 1 and 2 are essential to a theory of computation, we hence must give up item 3. Hence we really have to take partial functions seriously..

2.5. Exercises

Exercise 2.a. Think about how the variants of the definition of a Turing machine (listed after definition 2.1) are equivalent in the sense that one variant can be simulated in the other.

Exercise 2.b. Show that the following functions are Turing-computable by explicitly defining Turing machines a proving that they compute the respective function:

1. $f : \mathbb{N} \to \mathbb{N}, f(x) := 0$.
2. $f : \mathbb{N}^2 \to \mathbb{N}, f(x, y) := x + y$.
3. $f : \mathbb{N} \to \mathbb{N}, f(x) := 2y$.

It is helpful to first describe in plain English how the Turing machine program should work and then write down the actual table δ.

Exercise 2.c. Show that every finite partial function $f : \mathbb{N}^k \rightharpoonup \mathbb{N}$, i.e., one that is defined for only finitely many arguments, is computable.

Exercise 2.d. Show that the partial recursive functions are Turing-computable. Hint: Show that the collection $\mathcal{C}$ of Turing-computable functions satisfies the defining axioms of partial recursive functions (from definition 2.5).

Exercise 2.e. Show that primitive recursive functions are total.

Exercise 2.f. Provide an intuitively effective procedure that computes a function that is not primitive recursive. *Hint*: Argue that there must be a list $(f_n)_{n\in\mathbb{N}}$ of all primitive recursive functions, and then consider the 'diagonal' function $g(n) = f_n(n) + 1$.

Bonus: Think about why this argument does not work anymore once clause 6 is added. (Also see section 2.4.)

Exercise 2.g. Can you make the proof sketch of lemma 2.7 precise?

3. Incomputability

3.1. The Halting problem

3.2. The Entscheidungsproblem

3.3. Rice's theorem

3.4. Tiling problems

3.5. Exercises

4. Turing degrees: Ordering problems by difficulty

4.1. Computably enumerable sets

4.2. Reducibility

4.3. Turing degrees

4.4. Exercises

Part II.

Logic

Part III.

Randomness

A. Appendix: set theory and order theory

Set theory We use standard set-theoretic terminology as it is common in mathematics. (Sometimes, this is also called 'naive set theory' to distinguish it from the more advanced set theory studied in mathematical logic.)

To mention but one of many possible references for this: see Priest (2008, sec. 0.1).

A *set* is a collection of objects. We write $a \in A$ to say that object a is in (or is an element of, or is a member of) the set A. If $a_1, \ldots, a_n$ are objects, we write $\{a_1, \ldots, a_n\}$ for the set of these objects. Sets do not count 'order' and 'multiplicities', so $\{1, 0, 2, 2\} = \{0, 1, 2\}$. A set with just one element is called a *singleton*, and if a is an object, $\{a\}$ is the singleton of a (note $\{a\} \neq a$).

Here are some examples of specific sets. The set without any elements is called the *empty set* and is denoted $\emptyset$. The set of natural numbers is the set $\mathbb{N} = \{0, 1, 2, 3, \ldots\}$. (It is also denoted ω, because this is the ordinal notation from 'advanced' set theory.) The set of integers is written $\mathbb{Z}$, the set of rational numbers is written $\mathbb{Q}$, and the set of real numbers is written $\mathbb{R}$.

If A and B are sets, we say A is a *subset* of B (written $A \subseteq B$) if every element of A is an element of B. So the empty set trivially is a subset of any set. And two sets A and B are identical iff $A \subseteq B$ and $B \subseteq A$. If A and B are sets, then the *union* of A and B (written $A \cup B$) is the set containing exactly those objects that either are in A or in B (or both). The *intersection* of A and B (written $A \cap B$) is the set containing exactly those objects that are both in A and in B. The *complement* of a set A relative to a set B (written $B \setminus A$) is the set of objects that are in B but not in A. If B is clear from context, we just write A^c.

A *pair* (aka *ordered pair*) is a list of two elements (a, b); here the order matters, so $(a, b) \neq (b, a)$. (We can define (a, b) as the set $\{\{a\},\{a, b\}\}$.) More generally, an n-*tuple* is a list of n elements $(a_1, \ldots, a_n)$. Given n sets $A_1, \ldots, A_n$, their *Cartesian product* (written $A_1 \times \ldots \times A_n$) is the set of all n-tuples $(a_1, \ldots, a_n)$ such that, for all $i \in \{1, \ldots, n\}$, we have $a_i \in A_i$.

An n-ary *relation* between $A_1, \ldots, A_n$ is a subset of $A_1 \times \ldots \times A_n$. A 1-ary (resp., 2-ary, 3-ary) relation is also called a *unary* (resp., *binary*, *ternary*) relation. For a binary relation R, we usually write aRb instead of $(a, b) \in R$.

A *function* from a set A (its *domain*) to a set B (its *codomain*) is a binary relation f between A and B such that, for every $a \in A$, there is exactly one

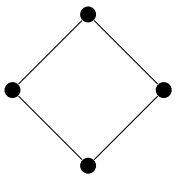


Figure A.1.: The Hasse diagram of $(\mathcal{P}(\{2,5\}),\subseteq)$.

$b \in B$ such that afb. We then write $f : A \to B$ and $f(a) = b$ or, if f is clear from context, $a \mapsto b$. By an n-*ary* function on a set A we mean a function $f : A^n \to A$, where $A^n = A \times \ldots \times A$ is the n-time Cartesian product of set A.

A *partial function* $f : A \rightharpoonup B$ is like a function $f : A \to B$ except that we only require that for every $a \in A$ there is at most one b such that afb. That is, for all elements $a \in A$, either a is mapped to one unique value b and we say f is defined on a, or a is not mapped to anything and we say f is undefined on a. We write $\mathsf{dom}(f)$ for the set of those $a \in A$ for which f is defined. Hence $\mathsf{dom}(f) \subseteq A$. If $\mathsf{dom}(f) = A$, i.e., f is defined on all elements of A, we say that f is *total*. Thus, 'total function' and 'function' are synonymous, but the former stresses that we are not just considering any partial function but in fact a total one.

Order theory Let S be a set and $\leqslant \subseteq S \times S$ a binary relation. We say $(S,\leqslant)$ is a *preorder* if $\leqslant$ is reflexive ($\forall x \in S : x \leqslant x$) and transitive ($\forall x,y,z \in S :$ $x \leqslant y$ and $y \leqslant z \Rightarrow x \leqslant z$). If $\leqslant$ is also antisymmetric ($\forall x,y \in S : x \leqslant$ y and $y \leqslant x \Rightarrow x = y$), we call $(S,\leqslant)$ a partial order. One defines $x < y$ as $x \leqslant y$ and $x \neq y$.

Finite partial orders can be represented by *Hasse diagrams*: the elements of S become points in the diagram, and there is a line from x to y if $x < y$ and there is no other element between x and y (i.e., there is no z with $x < z < y$). If this holds, one also says that y *covers* x. For example, the order $(\mathcal{P}(\{2,5\}),\subseteq)$ of the subsets of the two element set $\{2,5\}$ ordered by inclusion is described by the Hasse diagram of figure A.1. It often is very helpful to think of partial orders visually as Hasse diagrams. Since they only depict the covering relation, they are less cluttered, but from it one can recover the partial order as the reflexive and transitive closure of the covering relation (i.e., add $x \leqslant x$, and add $x \leqslant z$ whenever $x \leqslant y$ and $y \leqslant z$ is depicted).

There are various notions of bounds: Let $(S,\leqslant)$ be a preorder (which

includes partial orders). If $A \subseteq S$ is a subset, an *upper bound* (resp., *lower bound*) of A is an element $x \in S$ (if it exists) such that, for all $a \in A$, we have $x \geqslant a$ (resp., $x \leqslant a$). Moreover, x is the *least upper bound* (resp., *greatest lower bound*) of A if x is an upper bound (resp., lower bound) of A and, for all upper bounds (resp., lower bounds) y of A, we have $x \leqslant y$ (resp., $x \geqslant y$). Least upper bounds are also called *suprema*, and greatest lower bounds are also called *infima*. Note that, if least upper bounds exist, they are unique: i.e., if x and x' are least upper bounds of A, then $x = x'$—so we can speak of *the* least upper bound and also denote it $\bigvee A$. Similarly for greatest lower bounds, which we also denote $\bigwedge A$.

There are various notions of 'extremal' elements: Let $(S, \leqslant)$ be a preorder (which includes partial orders). An element $x \in S$ is the *greatest* (resp., *least*) element of S if, for all $s \in S$, $x \geqslant s$ (resp., $x \leqslant s$). Again, if existent, greatest and least elements are unique. An element $x \in S$ is a *maximal* (resp., *minimal*) element of S if, for all $s \in S$, if $x \leqslant s$ (resp., $s \leqslant x$), then $x = s$. So there is no other element strictly above (resp., below) x. Note that being maximal (resp., minimal) doesn't imply being greatest (resp., least): a partial order can have several maximal (resp., minimal) elements. (Think of some examples.)

Bibliography

Boolos, G. S., J. P. Burgess, and R. C. Jeffrey (2007). *Computability and Logic*. 5th ed. Cambridge: Cambridge University Press (cit. on p. 2).

Cooper, S. B. (2004). *Computability Theory*. Boca Raton,FL: Chapman & Hall/CRC (cit. on p. 2).

Copeland, B. J. (2026). "The Church-Turing Thesis." In: *The Stanford Encyclopedia of Philosophy*. Ed. by E. N. Zalta and U. Nodelman. Spring 2026. Metaphysics Research Lab, Stanford University (cit. on p. 16).

Cutland, N. (1980). *Computability: An Inroduction to recursive function theory*. Cambridge: Cambridge University Press (cit. on p. 2).

Downey, R. G. and D. R. Hirschfeldt (2010). *Algorithmic Randomness and Complexity*. New York: Springer (cit. on p. 2).

Gandy, R. (1980). "Church's Thesis and Principles for Mechanisms." In: *The Kleene Symposium*. Ed. by J. Barwise, H. J. Keisler, and K. Kunen. Amsterdam: North-Holland Publishing Company, pp. 123–148 (cit. on p. 9).

Gödel, K. (1946/1990). "Remarks before the Princeton bicentennial conference on problems in mathematics." In: *Kurt Gödel: Collected Works*. Ed. by S. Feferman et al. Vol. II. 1990. New York: Oxford University Press, pp. 150–153 (cit. on p. 18).

Icard, T. (11/2019). "Why Be Random?" In: *Mind* 130.517, pp. 111–139. DOI: 10.1093/mind/fzz065 (cit. on p. 9).

Marks, A. (2024). "Computability theory." Lecture notes from February 25, 2024. URL: https://math.berkeley.edu/~marks/notes/computability_notes_v1.pdf (cit. on pp. 2, 15).

Nies, A. (2009). *Computability and Randomness*. Oxford: Oxford University Press (cit. on p. 2).

Odifreddi, P. (1990). *Classical Recursion Theory: The Theory of Functions and Setsof Natural Numbers*. Studies in Logic and the Foundations ofMathematics, vol. 125. Amsterdam: North-Holland Publishing Company (cit. on pp. 2, 17 sq.).

Pour-El, M. B. and I. J. Richards (1989). *Computability in Analysis and Physics*. Perspectives in Mathematical Logic. Berlin, Heidelberg: Springer (cit. on p. 9).

Priest, G. (2008). *An Introduction to Non-classical Logic. From If to Is*. 2nd ed. Cambridge: Cambridge University Press (cit. on p. 27).

Robič, B. (2020). *The Foundations of Computability Theory*. Berlin Heidelberg: Springer. DOI: 10.1007/978-3-662-62421-0 (cit. on p. 2).

Rogers Jr., H. (1967). *Theory of Recursive Functions and Effective Computability*. New York: McGraw–Hill Book Company (cit. on p. 2).

Sieg, W. (2002). "Calculations by man and machine: conceptual analysis." In: *Reflections on the foundations of mathematics: Essays in honor of Solomon Feferman*. Ed. by W. Sieg, R. Sommer, and C. Talcott. Vol. 15. Lecture Notes in Logic. Natick, Massachusetts: Association for Symbolic Logic, pp. 390–409 (cit. on p. 9).

Smith, P. (2011). "Squeezing arguments." In: *Analysis* 71.1, pp. 22–30. DOI: 10.1093/analys/anq120 (cit. on p. 16).

Soare, R. I. (2016). *Turing Computability: Theory and Applications*. Berlin, Heidelberg: Springer. DOI: https://doi.org/10.1007/978-3-642-31933-4 (cit. on pp. 2, 18, 20).

Tourlakis, G. (2022). *Computability*. Cham: Springer. DOI: 10.1007/978-3-030-83202-5 (cit. on p. 2).

Yanofsky, N. S. (2022). *Theoretical Computer Science for the Working Category Theorist*. Cambridge Elements. Cambridge: Cambridge University Press (cit. on p. 2).

Zach, R. (2023). "Hilbert's Program." In: *The Stanford Encyclopedia of Philosophy*. Ed. by E. N. Zalta and U. Nodelman. Winter 2023. Metaphysics Research Lab, Stanford University (cit. on p. 4).